SuiviMat

Fonctionnement technique de l'application

Ce que fait chaque page, chaque script et chaque API, de l'interface jusqu'à la base de données.

État du code : branche `main`, commit `9d33930` du 4 novembre 2026
Technologies : PHP 8 (sans framework), MariaDB / MySQL, JavaScript sans bibliothèque, Bootstrap 5 + Sass, TCPDF
Comment lire ce document : les chapitres 1 à 3 posent les bases (architecture, conventions, base de données) ; le chapitre 4 suit chaque fonctionnalité de l'interface jusqu'au SQL ; l'annexe liste toutes les API.

Sommaire

1 Vue d'ensemble

1.1 Ce que fait l'application

1.2 Architecture

1.3 Arborescence des dossiers

1.4 Pages et API : deux façons de parler au serveur

2 Les fondations

2.1 config/init.php

2.2 Sessions et authentification

2.3 Classes et Managers

2.4 Anatomie d'une API

2.5 Anatomie d'une page

2.6 Anatomie d'un script

2.7 Sécurité : les règles appliquées partout

3 La base de données

3.1 Schéma

3.2 Les tables

3.3 Cycle de vie d'un objet

3.4 Migrations

4 Les fonctionnalités, de l'interface au SQL

4.1 Compte : inscription, connexion, Mon compte

4.2 Tableau de bord, organisations, invitations

4.3 Initialisation d'une organisation

4.4 Codes QR

4.5 L'inventaire

4.6 Catégories et champs personnalisés

4.7 Enregistrer des objets

4.8 Le scanner

4.9 La fiche d'un objet

4.10 Prêt, retour, dégâts

4.11 Vente, perte, remise à disposition

4.12 Code QR scanné et objet perdu

4.13 Prêts en cours et clients

4.14 Paramètres de l'organisation

4.15 Contrat PDF, export CSV, rappels

5 Le front-end transversal

5.1 Header, footer et mise en page

5.2 CSS : Bootstrap, Sass et variables

5.3 Scripts partagés

6 Annexes

6.1 Toutes les API

6.2 Mise en ligne

6.3 Limites connues

1 Vue d'ensemble

1.1 Ce que fait l'application

SuiviMat aide une association ou une petite entreprise à suivre son **matériel de prêt**. Chaque objet porte une étiquette avec un **code QR** unique de 4 caractères (ex. `V56G`). En scannant l'étiquette, un membre ouvre la fiche de l'objet et peut le prêter, enregistrer son retour, signaler des dégâts, le vendre ou le déclarer perdu.

Qui	Ce qu'il fait dans SuiviMat
Utilisateur (compte SuiviMat)	Se connecte, crée des organisations ou rejoint celles qui l'invitent. Dans une organisation, tous les membres ont les mêmes droits.
Organisation	Possède le matériel, les catégories, les clients et les réglages. Chaque organisation ne voit que ses propres données.
Client (emprunteur)	N'a **pas** de compte : c'est une fiche (prénom, nom, e-mail facultatif) gérée par l'organisation. Il peut recevoir un rappel par e-mail.
Visiteur	Personne non connectée qui scanne le code QR d'un objet **perdu** : il voit une page publique pour écrire à l'organisation.

1.2 Architecture

L'application est écrite en **PHP « classique »**, sans framework : chaque URL correspond à un fichier PHP. Il y a deux sortes de fichiers PHP appelés par le navigateur, et une couche de classes partagée.

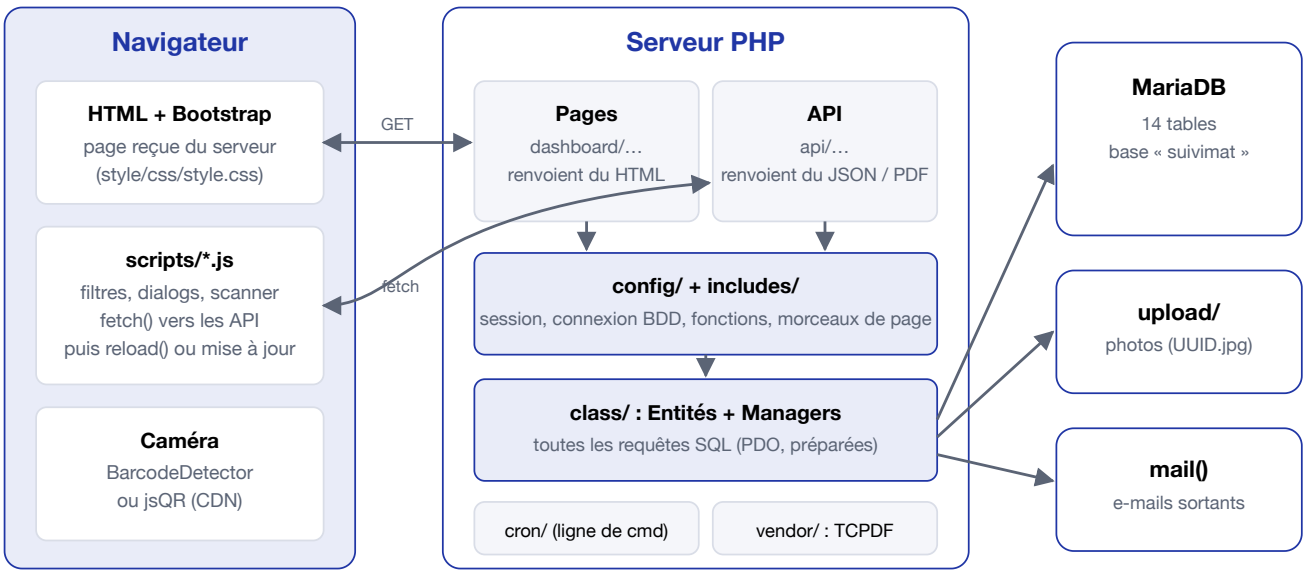


Figure 1 — Architecture générale. Les pages renvoient du HTML complet ; les API sont appelées en JavaScript et renvoient du JSON (ou un PDF / CSV).

Le principe le plus important : **aucune page ni API n'écrit de SQL pour les données métier** (à quelques exceptions de lecture près). Elles appellent des **Managers** (ex. `LoanManager`) qui contiennent les requêtes. C'est aussi là qu'est appliquée la règle « une organisation ne voit que ses données » (voir 2.3).

1.3 Arborescence des dossiers

Dossier	Contenu
`/` (racine)	Site public : `index.php`, `about.php`, `contact/`, `mentions-legales.php`.
`auth/`	Connexion (`login/`), inscription (`register/`), `logout.php`.
`dashboard/`	L'application : tableau de bord, `account/`, et `organisation/` (inventaire, `article/`, `loans/`, `clients/`, `settings/`, initialisation `init-orga.php`).
`api/`	Points d'entrée JSON, rangés par sujet : `account`, `article`, `category`, `client`, `invitation`, `loan`, `loss`, `organisation`, `sale`.
`class/`	Une paire **Entité + Manager** par sujet (ex. `loan/loan.php` + `loan/loanManager.php`).
`config/`	`init.php` (inclus partout), `session.php`, `auth.php` (pages réservées aux membres).
`includes/`	Morceaux réutilisés : `head.php`, headers, footers, `modals/`, formulaires partagés. **Bloqué** en accès direct par un `.htaccess`.
`scripts/`	Un fichier JS par comportement (ex. `article-loan.js` pour le dialog « Prêter »).
`style/`	`scss/style.scss` (source) → `css/style.css` (compilé avec `npm run build`).
`database/migrations/`	Scripts SQL à exécuter une fois, dans l'ordre (3.4).
`cron/`	Tâche planifiée des rappels (ligne de commande uniquement).
`upload/`	Photos envoyées. Exclu de Git (`.gitignore`).

1.4 Pages et API : deux façons de parler au serveur

Toute l'application repose sur ce va-et-vient. Exemple : ouvrir la fiche d'un objet, puis le prêter.

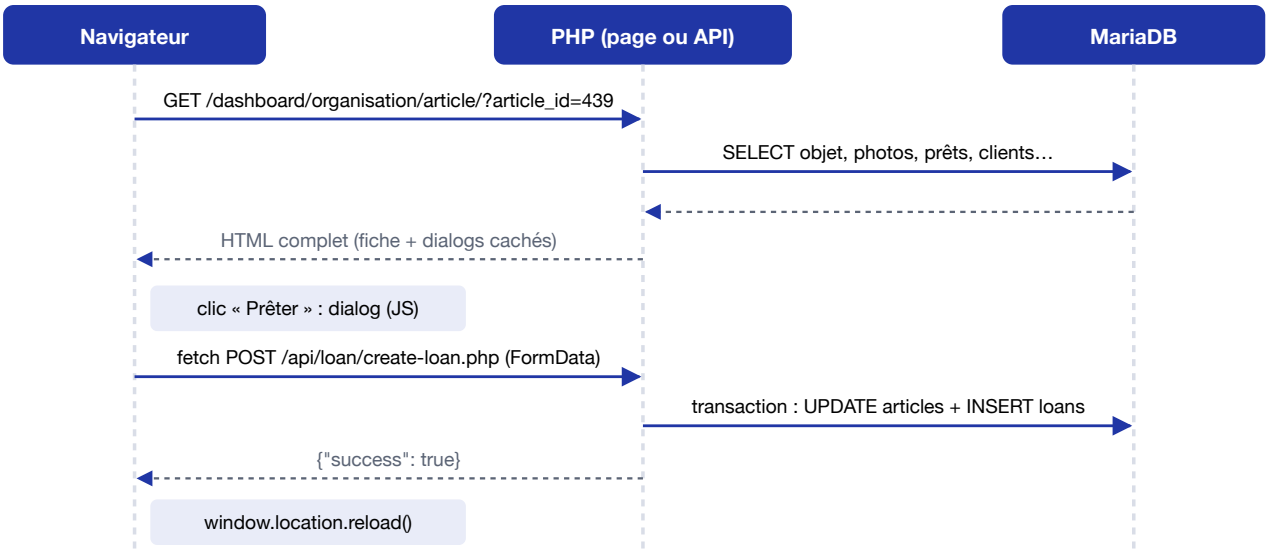


Figure 2 — Une page est générée en PHP (HTML) ; les actions passent par une API appelée en JavaScript, puis la page est rechargée.

Pourquoi recharger la page après une action ?

La fiche d'un objet affiche beaucoup d'informations liées (statut, historique, boutons disponibles). Plutôt que de tout mettre à jour en JavaScript, le script recharge la page : PHP la reconstruit avec l'état exact de la base. C'est plus simple et impossible à désynchroniser. Les listes filtrables (inventaire, prêts, clients), elles, se mettent à jour sans rechargement.

2 Les fondations

Ce chapitre décrit ce qui est utilisé par toutes les pages et API. Une fois ces conventions connues, chaque fichier se lit facilement.

2.1 config/init.php — inclus en premier partout

Chaque page et chaque API commence par `require_once …/config/init.php`. Ce fichier :

1. démarre la **session** (en incluant `config/session.php`) ;
2. ouvre la connexion à la base dans `$mysqlClient` (objet **PDO**, erreurs en exceptions). En local (`localhost`), identifiants écrits dans le fichier ; en ligne, lus dans les **variables d'environnement** `DB_HOST`, `DB_NAME`… ;
3. règle le fuseau sur **UTC** (PHP et MySQL) ;
4. définit les **fonctions utilitaires** globales et crée `$userManager`.

Fonction / constante	Rôle
`APP_VERSION`	Version affichée dans les deux footers.
`QR_PDF_DIRECTORY`	Dossier temporaire (hors du site) des PDF de codes QR générés (4.4).
`asset($path)`	Ajoute `?v=` + date de modification du fichier : `/scripts/x.js?v=1793800000`. Le navigateur ne garde jamais une ancienne version en cache.
`safeRedirect($path)`	Accepte une adresse de redirection seulement si elle est interne au site (refuse `https://…`, `//…`, `/\…`).
`formatDate($date)`	`2026-11-04` → `04.11.2026`.
`parseDate($text)`	Valide une date de formulaire (`AAAA-MM-JJ`) ; `null` si invalide (refuse le 30 février).
`parseOptionalEmail($text)`	`null` si vide, l'adresse si valide, `false` si invalide (type `string|false|null`).
`sendMail($to, $subject, $body, $replyTo)`	Envoie un e-mail avec `mail()`. **En local, n'envoie rien** : écrit le message dans le journal PHP.

2.2 Sessions et authentification

`config/session.php` règle le cookie de session avant `session_start()` : `httponly` (illisible en JavaScript), `secure` en HTTPS, `samesite = Strict` (jamais envoyé depuis un autre site, ce qui protège contre les requêtes forgées), durée de **12 heures** d'inactivité (`$_SESSION['last_activity']`).

`config/auth.php` est inclus par toutes les pages et API réservées aux membres :

config/auth.php

```php
if (!isset($_SESSION['user_id'])) {
    header('Location: /auth/login/?redirect=' . urlencode($_SERVER['REQUEST_URI']));
    exit();
}
$user = $userManager->getUserById($_SESSION['user_id']);           // utilisateur connecté
$invitationManager = new InvitationManager($mysqlClient, $user);
$organisationManager = new OrganisationManager($mysqlClient, $user);  // lié à CET utilisateur
```

Après ce fichier, une page dispose donc toujours de `$user`, `$organisationManager` et `$invitationManager`. Le paramètre `?redirect=` permet de revenir sur la page demandée après la connexion (4.1).

2.3 Classes et Managers

Chaque sujet a deux classes :

- une **entité** (`Loan`, `Client`…) : un objet simple avec des propriétés privées, une méthode `hydrate(array $ligne)` qui la remplit à partir d'une ligne SQL, et des getters ;
- un **Manager** (`LoanManager`…) qui fait toutes les requêtes et renvoie des entités.

`class/client/clientManager.php (extrait)`

```php
class ClientManager
{
    public function __construct(PDO $pdo, Organisation $organisation) { … }   // toujours une organisation

    public function getClientById(int $id): Client
    {
        $stmt = $this->pdo->prepare('SELECT * FROM clients WHERE id_client = :id AND idx_organisation = :organisation');
        $stmt->execute(['id' => $id, 'organisation' => $this->organisation->getId()]);
        …  // pas de ligne → Exception ; sinon new Client() + hydrate()
    }
}
```

La règle qui sépare les organisations

Les Managers de données (`ArticleManager`, `CategoryManager`, `ClientManager`, `LoanManager`, `SaleManager`, `LossManager`) reçoivent une `Organisation` à leur création et ajoutent `idx_organisation = :organisation` (ou une jointure avec `articles`) à **chaque** requête. Et cette organisation ne peut être obtenue qu'à travers `OrganisationManager`, qui vérifie que l'utilisateur connecté en est **membre**. Résultat : même en modifiant un identifiant dans l'URL, impossible de lire ou modifier les données d'une autre organisation.

Méthode de OrganisationManager	Retourne l'organisation…
`getOrganisationById($id)`	…d'après son ID (pages `?id=`).
`getOrganisationOfArticle($id)`	…d'un objet (fiche, API des objets, prêts, ventes, pertes).
`getOrganisationOfClient($id)`	…d'un client (page d'un client).
`getOrganisationOfLoan($id)`	…d'un prêt (contrat PDF).

`findByArticleCode($pdo, $code)` *(statique)*	…d'un code QR, **sans** vérifier l'appartenance : seule exception, pour la page publique d'un objet perdu.

Toutes lèvent une exception si l'utilisateur n'est pas membre ; les pages et API la transforment en erreur 404.

Manager	Méthodes principales
`ArticleManager`	`getArticleById`, `getArticleByCode`, `searchConfiguredArticles` (inventaire), `countConfiguredArticlesByCategory`, `configureArticle`, `createBlankArticles`, `getDisplayedFields`, `getAdditionalFieldValues`
`CategoryManager`	`getAllCategories`, `getCategoryPath`, `getDescendantIds`, `findCategoryIdsByName`, `createCategory`, `updateCategory`, `deleteCategory`, `getAdditionalFieldsByCategory`
`ImageManager`	`getArticleImages`, `getCoverUrls`, `getUploadedPhotos` (statique), `addArticleImages`, `deleteArticleImages`
`LoanManager`	`getLoansForArticle`, `getOngoingLoan`, `getOngoingLoans`, `getLoansForClient`, `getLoanById`, `createLoan`, `updateLoan`, `returnLoan`
`SaleManager` / `LossManager`	`sellArticle`, `cancelSale`, `getLatestSale` / `markLost`, `markFound`, `getCurrentLoss`
`ClientManager`	`getAllClients`, `getClientsWithLoanCounts`, `getClientById`, `createClient`, `updateClient`
`OrganisationManager`	les méthodes ci-dessus + `getAllOrganisationsForUser`, `createOrganisation`, `rename`, `updateLoanSettings`, `updateLostSettings`, `getMembers`, `removeMember`, `deleteOrganisation`
`UserManager` / `InvitationManager`	`getUserByEmail`, `createUser`, `updateUser`, `updateUserPassword`, `deleteUser`, `getEmailsOfOrganisation` / `invite`, `cancel`, `respond`, `getAllInvitationsForUser`

2.4 Anatomie d'une API

Toutes les API suivent le même déroulement. Voici `api/sale/cancel-sale.php`, une des plus courtes, annotée :

`api/sale/cancel-sale.php`

```php
require_once …'/config/init.php';          // 1. session + PDO + fonctions
require_once …'/config/auth.php';          // 2. connecté ? sinon redirection
header('Content-Type: application/json; charset=utf-8');

function sendError(int $status, string $message): void {   // 3. réponse d'erreur commune
    http_response_code($status);
    echo json_encode(['success' => false, 'message' => $message]);
    exit;
}
if ($_SERVER['REQUEST_METHOD'] !== 'POST') sendError(405, 'Méthode non autorisée');

$articleId = filter_input(INPUT_POST, 'article_id', FILTER_VALIDATE_INT);   // 4. lecture + validation
try {                                                                       // 5. appartenance
    $organisation = $organisationManager->getOrganisationOfArticle((int) $articleId);
    $article = (new ArticleManager($mysqlClient, $organisation))->getArticleById((int) $articleId);
} catch (Exception $e) { sendError(404, 'Objet introuvable'); }

try {                                                                       // 6. action (Manager)
    (new SaleManager($mysqlClient, $organisation))->cancelSale($article);
} catch (SaleNotCancellableException $e) {
    sendError(409, 'Cet objet n\'est plus vendu : rechargez la page');   // conflit attendu
} catch (Exception $e) {
    error_log(…); sendError(500, 'Erreur lors de l\'enregistrement');   // panne : détail au journal
}
echo json_encode(['success' => true]);                                      // 7. réponse
```

Code HTTP	Utilisé quand…
200	tout s'est bien passé (`{"success": true, …}`)
400	données invalides (date, champ vide, mot de passe trop faible…)
403	mot de passe actuel incorrect, ou script réservé à la ligne de commande
404	élément introuvable **ou appartenant à une autre organisation** (on ne révèle pas qu'il existe)
405	mauvaise méthode (GET au lieu de POST)
409	conflit : l'état a changé entre-temps (objet déjà prêté, vente déjà annulée…)
413 / 429	photos trop lourdes / trop de messages envoyés (page objet perdu)
500	erreur inattendue (le détail est écrit dans le journal, jamais envoyé au navigateur)

Les fichiers de validation partagés

Quand plusieurs API lisent les mêmes champs, la lecture est dans un fichier de `includes/` inclus avec `require`. Un fichier inclus partage les variables du fichier qui l'inclut : après l'inclusion, l'API dispose des valeurs déjà vérifiées.

Fichier	Lit et vérifie	Utilisé par
`includes/client-form.php`	client existant (`client_id`) ou nouveau (`new` + prénom, nom, e-mail)	prêt, modification de prêt, vente

`includes/loan-form.php`	client + date de sortie + retour prévu (après la sortie)	`create-loan` , `update-loan`
`includes/category-fields.php`	champs personnalisés (nom, valeur par défaut, doublons, 20 max)	`create-category` , `update-category`

2.5 Anatomie d'une page

Une page de l'application est un fichier PHP qui, dans l'ordre : vérifie les droits, lit les données avec les Managers, puis écrit le HTML en assemblant des morceaux de `includes/` .

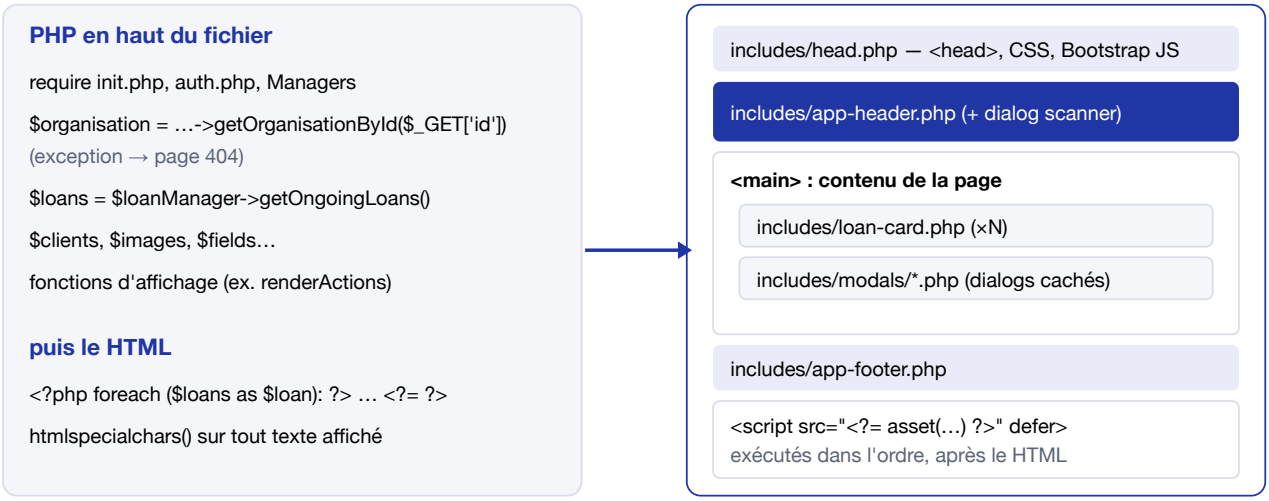


Figure 3 — Construction d'une page : données lues en PHP, puis assemblage du HTML avec des morceaux partagés.

Le **site public** (accueil, connexion…) utilise `includes/header.php` et `includes/footer.php` ; l'**application** utilise `includes/app-header.php` et `includes/app-footer.php` avec `<body class="app">` (5.1).

> **Passer des données du PHP au JavaScript**
>
> Deux techniques sont utilisées :
>
> - **attributs `data-*`** sur un élément : `<form data-article-id="439" data-loan-start="2026-11-01">` , lus en JS par `form.dataset.articleId` ;
> - **bloc JSON** pour les données volumineuses (dialog « Modifier l'objet ») : `<script type="application/json" id="article-edit-data"><?= json_encode($editData, JSON_HEX_TAG | JSON_HEX_AMP) ?></script>` , lu par `JSON.parse(…textContent)` . `JSON_HEX_TAG` empêche un nom contenant `</script>` de fermer la balise.

2.6 Anatomie d'un script

Chaque script est enveloppé dans une fonction exécutée immédiatement (`(() => { … })()`) pour que ses variables ne soient pas globales. Il retrouve ses éléments par **`data-role`** (jamais par des classes de style), et suit toujours le même schéma :

`scripts/article-unsell.js (structure)`

```
(() => {
    const modal = document.getElementById("article-unsell-modal");
    if (!modal) return;                          // dialog absent de cette page : rien à faire
    const form = modal.querySelector("[data-role='unsell-form']");

    modal.addEventListener("show.bs.modal", () => { … });   // remise à zéro à chaque ouverture

    form.addEventListener("submit", async (e) => {
        e.preventDefault();                                 // pas d'envoi « classique » du
formulaire
        const formData = new FormData();
        formData.append("article_id", form.dataset.articleId);
        const response = await fetch("/api/sale/cancel-sale.php", { method: "POST", body:
formData });
        const result = await response.json();
        if (!result.success) { showMessage("danger", result.message); return; }
        window.location.reload();                           // la page se reconstruit avec le
nouvel état
    });
})();
```

- **Validation** : les formulaires ont `novalidate` ; le script appelle `form.checkValidity()` et ajoute la classe Bootstrap `was-validated` pour colorer les champs. Les règles impossibles en HTML (« retour après la sortie ») passent par `input.setCustomValidity("…")`.
- **Ouverture d'un dialog avec un contexte** : dans l'événement `show.bs.modal`, `e.relatedTarget` est le bouton qui l'a ouvert. Le dialog « Prêter » s'en sert pour passer en mode « modification » quand le bouton porte `data-loan-id`.
- **Communication entre scripts** par événements personnalisés : `code-scanned` (scanner), `article-configured`, `categories-changed`, `article-setup-add-code`, `qr-codes-generated`.

2.7 Sécurité : les règles appliquées partout

Risque	Protection dans SuiviMat
Injection SQL	Uniquement des requêtes préparées PDO (`:nom` ou `?`). Les seules valeurs insérées directement sont des entiers validés (`LIMIT`) ou des listes de `?` générées.
Injection HTML (XSS)	`htmlspecialchars()` sur tout texte affiché en PHP ; `textContent` (jamais `innerHTML` avec des données) en JavaScript.
Accès à une autre organisation	Managers liés à une organisation vérifiée (2.3).
Requêtes forgées depuis un autre site	Cookie de session `SameSite=Strict`.
Vol / fixation de session	`session_regenerate_id(true)` à la connexion et après un changement de mot de passe.
Redirection vers un faux site	`safeRedirect()` sur le paramètre `?redirect=`.
Deux actions simultanées	Transactions + condition sur le statut dans l' `UPDATE` (… `WHERE art_status = 'Disponible'`) puis vérification de `rowCount()` (4.10).
Fichiers envoyés	Vérification `getimagesize()` , ré-encodage en JPEG (supprime les métadonnées, dont la position GPS), nom aléatoire (UUID).

Spam (page publique)	Champ piège invisible, un message par minute et par session, uniquement pour un objet réellement perdu.
Mots de passe	`password_hash()` / `password_verify()` ; règles : 8 caractères, majuscule, minuscule, chiffre, caractère spécial.

3 La base de données

3.1 Schéma

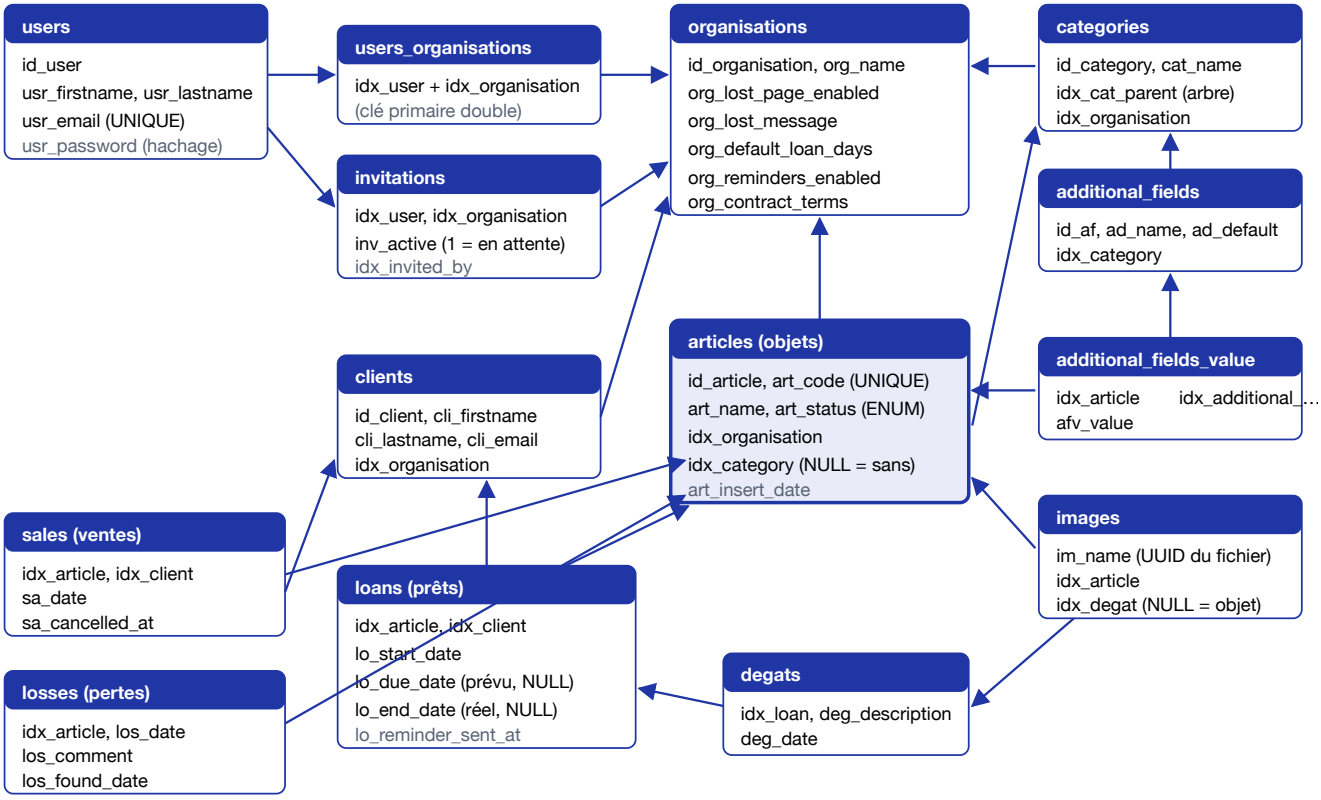


Figure 4 — Schéma simplifié. Une flèche va d'une table vers celle qu'elle référence (clé étrangère). La table centrale est `articles`.

> **Conventions de nommage**
>
> Les clés primaires s'appellent `id_…` ; les clés étrangères `idx_…` ; chaque colonne a le préfixe de sa table (`art_`, `lo_`, `cli_`…). **La table `loans` n'a pas de colonne d'organisation** : on la retrouve en joignant `articles`, d'où les `JOIN articles … WHERE articles.idx_organisation = :organisation` dans `LoanManager`. Même principe pour `sales`, `losses`, `degats` et `images`.

3.2 Les tables

Table	Rôle et colonnes à connaître
`articles`	Un objet = un code QR. `art_code` : 4 caractères uniques sur tout le site. `art_status` : `ENUM('Non attribué','Disponible','Emprunté','Vendu','Perdu')`. Un code généré mais pas encore configuré est « Non attribué » et n'apparaît pas dans l'inventaire.
`categories`	Arbre de catégories (`idx_cat_parent`). Les champs personnalisés d'une catégorie s'appliquent aussi à ses sous-catégories (héritage).
`additional_fields` / `…_value`	Champs personnalisés d'une catégorie (nom + valeur par défaut) et leurs valeurs pour chaque objet. Une valeur absente = on affiche la valeur par défaut.

`loans`	Un prêt. `lo_end_date` vide = **en cours**. `lo_due_date` vide = retour indéterminé.
`degats`	Dégâts constatés au retour d'un prêt ; leurs photos sont dans `images` avec `idx_degat`.
`sales` / `losses`	Historique des ventes (annulables) et des pertes (avec date de retrouvaille).
`clients`	Emprunteurs et acheteurs d'une organisation.
`users`, `users_organisations`, `invitations`	Comptes, appartenances (tous les membres ont les mêmes droits), invitations en attente.

3.3 Cycle de vie d'un objet

Le statut d'un objet (`art_status`) ne change que par des méthodes précises des Managers. Chaque transition vérifie le statut de départ dans la requête elle-même.

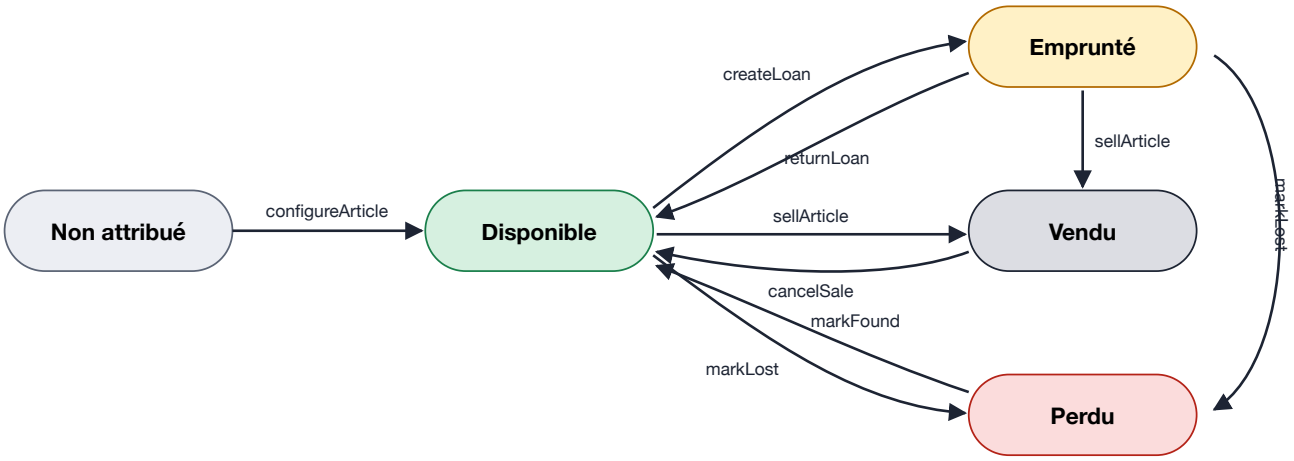


Figure 5 — Statuts et méthodes qui les changent. Vendre ou perdre un objet emprunté clôture automatiquement le prêt en cours.

3.4 Migrations

Le dossier `database/migrations/` contient les modifications de la base à appliquer **une seule fois**, dans l'ordre du nom :

Fichier	Effet
`2026-11-04-art-status-enum.sql`	Statuts en `ENUM` (liste fermée), anciens libellés convertis.
`2026-11-04-client-email.sql`	`clients.cli_email`.
`2026-11-04-degats-date.sql`	`deg_date` devient une vraie date.
`2026-11-04-field-default.sql`	Valeur par défaut des champs personnalisés.
`2026-11-04-loans-due-date.sql`	Date de retour prévue.
`2026-11-04-losses.sql`	Table des pertes.
`2026-11-04-organisation-settings.sql`	Réglages des organisations, rappels, invitations réparées.

`2026-11-04-sales.sql` puis `…-sales-cancel.sql`	Table des ventes, puis annulation des ventes.

Ordre réel

L'ordre alphabétique place `sales-cancel.sql` **avant** `sales.sql` (le « - » est classé avant le « . »). Or `sales-cancel.sql` modifie la table créée par `sales.sql` : exécutez `sales.sql` en premier. Les autres fichiers sont indépendants entre eux.

4 Les fonctionnalités, de l'interface au SQL

Pour chaque fonctionnalité : ce que voit l'utilisateur, le script qui réagit, l'API appelée, la méthode du Manager et les requêtes SQL. Les chaînes « Interface → … → SQL » résument le chemin complet.

4.1 Compte : inscription, connexion, Mon compte

Inscription

INTERFACE	JS	API	CLASSES	SQL
`auth/register/` : prénom, nom, e-mail, mot de passe ×2, reCAPTCHA	script dans la page : regex du mot de passe, puis `sendForm()`	`auth/register/submit-register.php`	`User::setPassword()`, `UserManager::createUser()`	`INSERT INTO users`

Le serveur vérifie les champs, applique les règles du mot de passe (`User::setPassword()` lève une `InvalidArgumentException` avec un message en français), interroge Google (`https://www.google.com/recaptcha/api/siteverify` avec la clé `RECAPTCHA_SECRET_KEY`), refuse une adresse déjà utilisée, crée le compte puis **connecte** directement l'utilisateur (`$_SESSION['user_id']`).

Connexion

INTERFACE	JS	API	CLASSES	SQL
`auth/login/` (Entrée ou bouton)	`sendForm(form, "submit-login.php")` (`scripts/sendForm.js`)	`auth/login/submit-login.php`	`getUserByEmail()`, `passwordVerify()`	`SELECT * FROM users WHERE usr_email = :email`

`auth/login/submit-login.php (fin)`

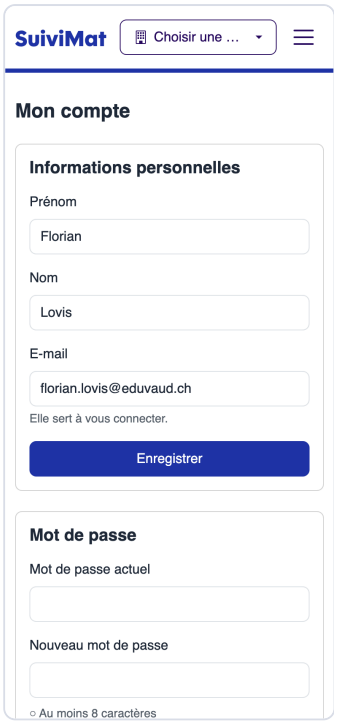
```php
if ($user === null || !$user->passwordVerify($password)) {   // même message dans les deux cas
    http_response_code(401); echo json_encode([…'Email ou mot de passe incorrect']); exit;
}
session_regenerate_id(true);                 // nouvel identifiant de session
$_SESSION['user_id'] = $user->getId();
echo json_encode(['success' => true, 'redirect' => safeRedirect($_POST['redirect'] ?? null)]);
```

`sendForm()` envoie le formulaire, affiche le message et suit `redirect`. Le champ caché `redirect` du formulaire contient la page demandée avant la connexion (ajoutée par `config/auth.php`), par exemple la fiche d'un objet dont on a scanné le code. La **déconnexion** (`auth/logout.php`) détruit la session et renvoie vers la connexion.

Mon compte

Page `dashboard/account/`, script `scripts/account.js`. Chaque carte est un formulaire `[data-role='account-form']` avec son API dans `data-url`.

Carte	API → méthode
Informations	`update-profile.php` → `UserManager::updateUser()`. Changer d'e-mail exige le mot de passe actuel (le champ apparaît en JS). Doublon d'e-mail : erreur MySQL 1062 → `EmailAlreadyUsedException` → 409.
Mot de passe	`update-password.php` → `updateUserPassword()` (vérifie l'actuel, puis `setPassword()`). Les 5 règles se cochent en direct : leurs expressions régulières sont écrites en PHP dans `data-rule` et testées par `new RegExp()`.
Supprimer	`delete-account.php` → `deleteUser()` : invitations, appartenances, compte. Refusé si l'utilisateur est seul membre d'une organisation (`getSoleMemberOrganisationNames()`).



Mon compte (mobile)

4.2 Tableau de bord, organisations, invitations

`dashboard/index.php` liste les organisations de l'utilisateur (`getAllOrganisationsForUser()`, triées par `org_last_open` : la dernière ouverte en premier) et ses invitations.

Créer une organisation

Dialog → `sendForm()` → `api/organisation/create-organisation.php` → `createOrganisation()` : dans une transaction, `INSERT INTO organisations` puis `INSERT INTO users_organisations`. Le script redirige ensuite vers l'initialisation (4.3).

Inviter et rejoindre

Un membre invite une adresse depuis les paramètres (4.14) : la personne doit avoir un compte. Sur son tableau de bord, `scripts/invitations.js` envoie `action = accept | reject` à `api/invitation/respond-invitation.php` → `InvitationManager::respond()` :



Tableau de bord

```
-- verrouille l'invitation jusqu'à la fin de la transaction : deux clics ne l'acceptent pas deux fois
SELECT idx_organisation FROM invitations WHERE id_invitation = :id AND idx_user = :user AND inv_active = 1 FOR UPDATE;
INSERT IGNORE INTO users_organisations (idx_user, idx_organisation) VALUES (:user, :organisation);  -- si accept
UPDATE invitations SET inv_active = 0 WHERE id_invitation = :id;
```

4.3 Initialisation d'une organisation

`dashboard/organisation/init-orga.php?id=…&page=1…4` est un parcours en 4 étapes. Le fichier principal inclut l'étape demandée (`init-orga/page1.php` … `page4.php`) ; chaque étape réutilise un module déjà présent ailleurs :

Étape	Contenu	Module réutilisé
1	Télécharger les codes QR	`includes/modals/download-qr-modal.php` + marche à suivre (`qr-next-steps-modal.php`)
2	Créer les catégories	`includes/category-setup.php` (4.6)
3	Enregistrer les objets	`includes/article-setup.php` (4.7)
4	Ajouter des clients	`includes/client-setup.php` + `scripts/client-setup.js` → `api/client/create-client.php`

4.4 Codes QR

INTERFACE	JS	API	CLASSES	SQL
dialog « Télécharger des codes QR » (format de feuille, nombre)	`download-qr-modal.js` : `fetch`, puis ouvre l'adresse reçue dans un onglet	`generate-qr-codes.php` → JSON `{ url }` ; `qr-pdf.php` sert le PDF	`createBlankArticles()`, TCPDF	`INSERT INTO articles (…, 'Non attribué')`

Génération et ouverture du PDF

Le script ouvre un onglet vide **pendant le clic** (ouvert après l'attente, il serait bloqué comme fenêtre surgissante), puis envoie le formulaire. L'API ne renvoie pas le PDF lui-même mais une adresse : un PDF reçu par `fetch` ne peut être affiché que depuis une adresse `blob:`, qui ne porte pas de nom de fichier, et le navigateur l'enregistrerait sous « document.pdf ».

1. `generate-qr-codes.php` écrit le PDF dans un fichier temporaire (`$pdf->Output($chemin, 'F')`) du dossier `QR_PDF_DIRECTORY` (dossier temporaire du système, hors du site), sous un nom aléatoire de 32 caractères (`bin2hex(random_bytes(16))`) ; les PDF de plus d'un jour sont supprimés au passage.
2. Il retient dans la session ce jeton et le nom du fichier : `$_SESSION['qr_pdfs'][$token] = 'etiquettes-qr-test-2026-11-04-1932.pdf'` (nom de l'organisation sans accents grâce à `iconv('UTF-8', 'ASCII//TRANSLIT')`), et répond `{ "success": true, "url": "/api/organisation/qr-pdf.php?token=…" }`.
3. Le script envoie l'onglet à cette adresse. `qr-pdf.php` vérifie que le jeton appartient à la session et ne contient que des caractères hexadécimaux (sinon `../` permettrait de sortir du dossier), puis envoie le

fichier avec `Content-Disposition: inline; filename="…"` : **inline** = affiché dans l'onglet, **filename** = nom proposé à l'enregistrement.

L'événement `qr-codes-generated` transmet cette adresse (`detail.pdfUrl`) : l'initialisation et l'enregistrement d'objets l'utilisent pour le lien de secours « Ouvrir le PDF ».

Nouveaux codes : `createBlankArticles($n)` tire des codes de 4 caractères dans un alphabet sans caractères ambigus (`ABCDEFGHJKMNPQRSTUVWXYZ23456789` : ni 0/O, ni 1/I/L) et les insère. Si un code existe déjà, MySQL refuse le doublon (erreur 1062, colonne `UNIQUE`) et on en tire un autre. Tout est dans une transaction. Le PDF contient le logo, le code QR et le code lisible de chaque étiquette.

Réimpression (`mode=reprint`) : aucun code n'est créé ; on réimprime soit des codes donnés (vérifiés avec `getArticleByCode()`), soit tous les objets enregistrés. Depuis la fiche d'un objet, « Réimprimer l'étiquette » ouvre le même dialog avec `data-reprint-code`.

> **Adresse contenue dans un code QR**
>
> `https://<site>/dashboard/organisation/article/?code=V56G`. N'importe quel appareil photo de téléphone l'ouvre ; c'est la fiche de l'objet qui décide quoi afficher selon la personne (4.12). Le scanner intégré lit aussi bien cette adresse qu'un code saisi à la main.

4.5 L'inventaire

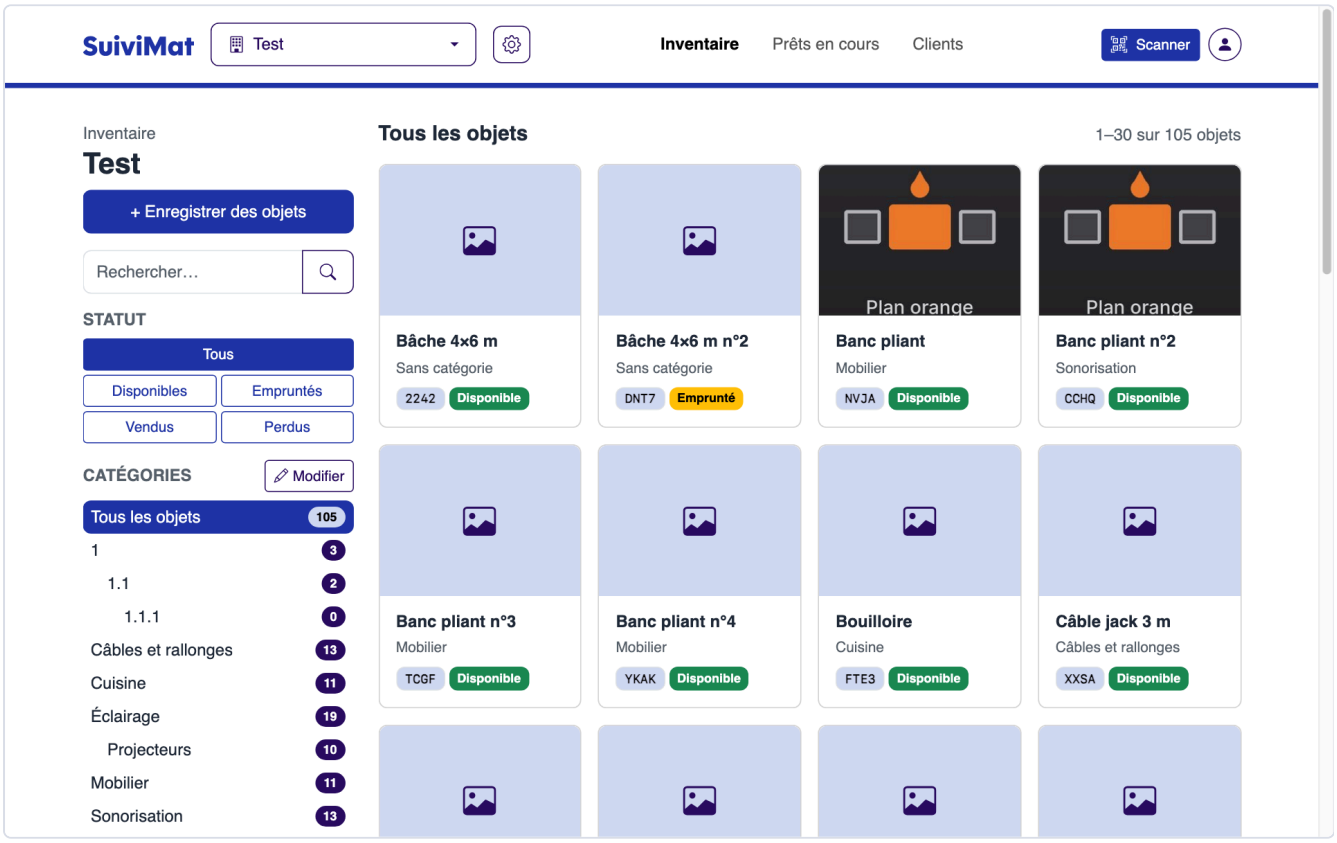


Figure 6 — Inventaire sur ordinateur : colonne de recherche et de filtres collante à gauche, cartes à droite.

`dashboard/organisation/index.php` ne contient presque aucune donnée : la page affiche une structure vide, et `scripts/organisation-articles.js` charge les catégories puis les objets par `fetch`. Le script garde un petit **état** :

```
let categories = [];   // liste des catégories (arbre)
let counts = {};       // nombre d'objets par catégorie, fourni par l'API
let selected = ALL;    // "all", "none" (sans catégorie) ou l'ID d'une catégorie
let status = ALL;      // "all", "disponible", "emprunte", "vendu", "perdu"
let query = "";       // recherche validée (Entrée)
let page = 1;
```

Chaque changement (catégorie, statut, page, recherche) modifie l'état puis appelle `loadArticles()` :

JS	API	MANAGER	IMAGES	RÉPONSE
`loadArticles()` construit les paramètres	`GET api/article/get-articles.php?organisation_id&page&categorie&statut&q`	`searchConfiguredArticles()` + `countConfiguredArticlesByCategory()`	`getCoverUrls()` : 1re photo de chaque objet	`articles, total, page, pages, counts, inventory_size`

La requête de l'inventaire

`searchConfiguredArticles()` construit la clause `WHERE` morceau par morceau, selon les filtres actifs :

`class/article/articleManager.php — searchConfiguredArticles() (simplifié)`

```
$where = ["idx_organisation = :organisation", "art_status <> 'Non attribué'"];
if ($status !== null)      $where[] = 'art_status = :status';
if ($withoutCategory)       $where[] = 'idx_category IS NULL';
elseif ($categoryIds)        $where[] = 'idx_category IN (:category0, :category1, …)';  // catégorie + sous-catégories
foreach ($words as $i => $word)                                  // CHAQUE mot doit correspondre
    $where[] = "(art_name LIKE :word$i OR art_code LIKE :word$i OR idx_category IN (…))";

SELECT COUNT(*) FROM articles WHERE …                              -- total, pour la pagination
SELECT * FROM articles WHERE … ORDER BY art_name, art_code LIMIT 30 OFFSET (page-1)*30
```

- Un mot trouve un objet par son **nom**, son **code**, ou le nom d'une de ses **catégories** (y compris parentes : `findCategoryIdsByName()` ajoute les sous-catégories).
- Les caractères `%` et `_` tapés par l'utilisateur sont échappés (`escapeLike()`) pour être cherchés tels quels.
- La collation de la base ignore la casse et les accents : « eclairage » trouve « Éclairage ».

Filtres, pagination et adresse

- **Catégories** (colonne de gauche) : arbre dessiné par `renderCategories()` avec, pour chacune, le nombre d'objets (`countIn()` additionne les sous-catégories). Sur mobile, la colonne devient un panneau latéral (`offcanvas` Bootstrap).
- **Statut** : boutons radio stylés (`btn-check`) ; les compteurs de catégories suivent le statut choisi.
- **Pages** de 30 objets : `pageNumbers()` affiche la première, la dernière et deux pages autour de la courante, avec « … ».
- **Adresse** : `updateUrl()` écrit l'état dans l'URL (`?statut=…&categorie=…&q=…&page=…`) avec `history.replaceState()` ; `readUrl()` le relit au chargement. Recharger ou partager le lien redonne la même vue.
- **Réponses dans le désordre** : chaque requête reçoit un numéro (`articlesRequest`) ; une réponse plus ancienne que la dernière demandée est ignorée.

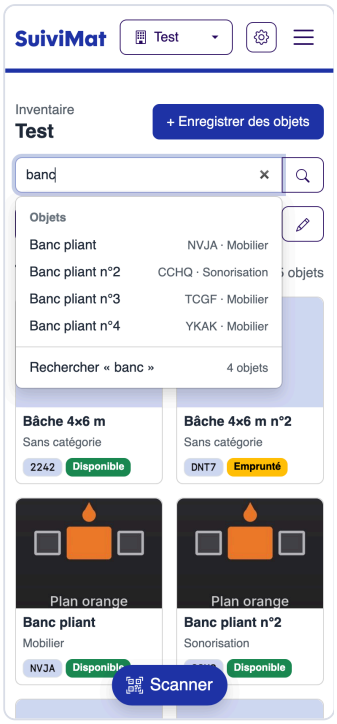
Recherche et propositions

Pendant la saisie, après 200 ms sans frappe, le script appelle `api/article/search-suggestions.php?q=…` qui renvoie au plus 3 catégories et 6 objets. Une requête encore en cours est annulée par un `AbortController` quand on tape à nouveau.

La liste suit le modèle d'accessibilité « combobox » : flèches haut/bas pour se déplacer, Entrée pour choisir, Échap pour fermer.

- choisir une **catégorie** → filtre l'inventaire ;
- choisir un **objet** → ouvre directement sa fiche ;
- valider une recherche qui ne trouve qu'**un seul objet** → ouvre sa fiche (`loadArticles({ openSingle: true })`) ; l'URL de la recherche est enregistrée avant, pour que « retour » ramène à la liste.

La colonne de gauche reste collée à l'écran (`position: sticky`) ; sa hauteur est mesurée par un `ResizeObserver` et transmise au CSS (`--sidebar-height`) pour qu'une longue liste de catégories défile avec la page sans barre de défilement propre.



Propositions de recherche

4.6 Catégories et champs personnalisés

Module `includes/category-setup.php` + `scripts/category-setup.js`, utilisé dans l'initialisation, dans le dialog « Modifier les catégories » de l'inventaire et dans les paramètres. Les API : `get-categories`, `create-category`, `update-category`, `delete-category`.

- Chaque champ est envoyé sous forme de tableau imbriqué : `fields[12][name]`, `fields[12][default]` → PHP reçoit `$_POST['fields'][12]['name']`. Les champs existants ont leur ID comme clé, les nouveaux `new_fields[…]`.
- `updateCategory()` (transaction) : renomme, change de parente, supprime les champs retirés **et leurs valeurs**, met à jour les autres, ajoute les nouveaux. Une catégorie ne peut pas devenir l'enfant d'une de ses sous-catégories.
- Suppression refusée si la catégorie contient des sous-catégories ou des objets.
- Après chaque modification, le script émet `categories-changed` : les formulaires d'objet ouverts mettent à jour leur liste.

Héritage et valeurs par défaut

Un objet de « Tentes › Igloo » a les champs de « Tentes » **et** d'« Igloo ». Côté navigateur, `scripts/category-fields.js` calcule cette liste (`window.categoryFields(id, categories)`) ; côté serveur, `configureArticle()` n'accepte que les champs de la catégorie et de ses parentes. Pour l'affichage, `getDisplayedFields()` part des **champs** et non des valeurs :

```
ArticleManager::getDisplayedFields()
```

```
SELECT additional_fields.ad_name, additional_fields.ad_default,
additional_fields_value.afv_value
FROM additional_fields
LEFT JOIN additional_fields_value                       -- garde aussi les champs sans valeur
    ON additional_fields_value.idx_additional_fields = additional_fields.id_af AND
additional_fields_value.idx_article = ?
WHERE additional_fields.idx_category IN (?, ?)          -- catégorie + parentes
ORDER BY FIELD(additional_fields.idx_category, ?, ?), additional_fields.id_af   -- parentes
d'abord
```

Valeur présente → affichée ; sinon valeur par défaut, avec la mention « (par défaut) ». Dans les formulaires, la valeur par défaut est montrée en `placeholder` et jamais copiée dans l'objet : si elle change dans la catégorie, tous les objets sans valeur propre suivent.

4.7 Enregistrer des objets

Module `includes/article-setup.php` + `scripts/article-setup.js`, dans un dialog de l'inventaire et à l'étape 3 de l'initialisation. Il combine un **scanner** et une **file d'attente** : on scanne plusieurs étiquettes, puis on configure les objets un par un.

SCAN	JS	FILE	ENVOI	SQL
événement `code-scanned`	`addCode()` : `GET api/article/get-article.php?code=`	l'objet rejoint `queue` ; le 1er est affiché	`POST api/article/configure-article.php` (nom, catégorie, `fields[]`, `photos[]`)	`UPDATE articles`, `DELETE` + `INSERT` valeurs, `INSERT images`

- Un objet déjà configuré peut être rescanné : il s'affiche avec le badge « déjà configuré » et ses valeurs.
- **Photos** : `scripts/resize-photo.js` (`window.resizePhoto()`) réduit chaque photo à 1600 px dans un `<canvas>` et la convertit en JPEG **avant l'envoi** (moins de données mobiles). Le serveur la vérifie, la ré-encode (orientation EXIF appliquée, métadonnées supprimées) et l'enregistre sous `upload/<UUID>.jpg`.
- `configureArticle()` (transaction) passe un objet « Non attribué » à « Disponible » (`IF(art_status = 'Non attribué', 'Disponible', art_status)`) et remplace toutes ses valeurs de champs.
- Après chaque objet, l'événement `article-configured` fait recharger la liste de l'inventaire.

4.8 Le scanner

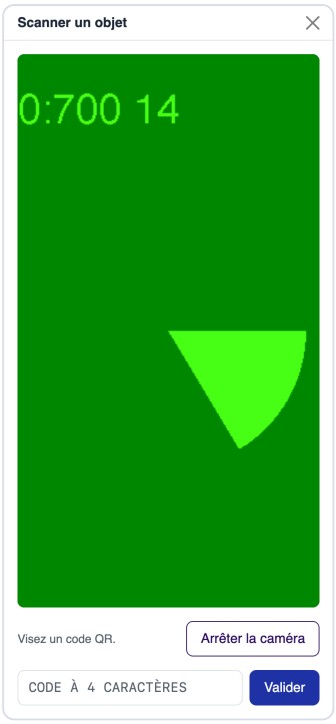
`includes/scanner.php` + `scripts/scanner.js` : une classe `Scanner` par élément `[data-scanner]` (il peut y en avoir deux par page : `element.scanner ??= new Scanner(element)` évite de les créer deux fois).

1. `start()` : `navigator.mediaDevices.getUserMedia({ video: { facingMode: "environment" } })` (caméra arrière ; HTTPS ou localhost requis).
2. Toutes les 200 ms, l'image vidéo est copiée dans un `<canvas>` réduit à 640 px, puis analysée par le détecteur natif **BarcodeDetector** s'il existe, sinon par **jsQR**, chargé à la demande depuis jsDelivr (avec contrôle d'intégrité `integrity`).
3. `parseCode()` accepte une adresse (`…?code=V56G`) ou le code seul, et vérifie `/^[A-Z0-9]{4}$/`.
4. Le même code est ignoré pendant 3 s ; sinon vibration, cadre vert et événement `code-scanned` (`detail: { code, source, scanner }`). Le champ de saisie manuelle émet le même événement.

Le scanner rapide

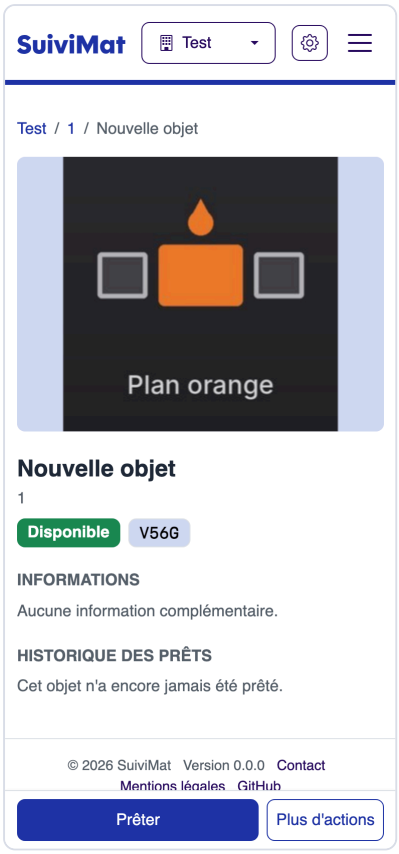
Le bouton « Scanner » (header sur ordinateur, bouton flottant sur l'inventaire mobile, menu burger) ouvre `quick-scan-modal.php`, inclus par le header. `scripts/quick-scan-modal.js` allume la caméra à l'ouverture, puis pour chaque code appelle `get-article.php` :

- objet configuré → `/dashboard/organisation/article/?article_id=…` ;
- code « Non attribué » → ouvre « Enregistrer des objets » avec ce code (ou, sur une page sans ce dialog, l'inventaire avec `?enregistrer=CODE`).



Scanner rapide (image de la fausse caméra de test de Chrome)

4.9 La fiche d'un objet



Mobile : actions dans une barre fixe en bas



Dialog « Prêter »

`dashboard/organisation/article/index.php` est la page la plus riche. En haut du fichier, PHP lit tout ce qui est nécessaire :

```
$categoryPath = $categoryManager->getCategoryPath($article->getCategoryId());  // [Tentes, Igloo] : chemin d'accès
$images       = $imageManager->getArticleImages($article);                      // carrousel
$fields       = $articleManager->getDisplayedFields($article);                  // Informations
$loans        = $loanManager->getLoansForArticle($article);                     // historique + dégâts
$sale         = … getLatestSale($article);   $loss = … getCurrentLoss($article);
$clients      = $clientManager->getAllClients();                               // dialogs Prêter / Vendre
$editData     = [ … ];                                                         // JSON pour « Modifier »
```

Puis elle écrit : chemin d'accès (chaque catégorie ouvre l'inventaire filtré), carrousel Bootstrap, nom, statut, informations, historique, et **tous les dialogs** dont l'objet peut avoir besoin selon son statut. Les boutons d'action sont produits par la fonction `renderActions($status, $dropup, $code)`, appelée deux fois : barre fixe en bas sur mobile (`fixed-bottom d-lg-none`), sous le nom sur ordinateur.

Statut	Bouton principal	« Plus d'actions »
Disponible	Prêter	Modifier, Réimprimer l'étiquette, Marquer comme vendu, Marquer comme perdu
Emprunté	Enregistrer un retour	Modifier, Réimprimer, Vendu, Perdu

Vendu	Remettre disponible	Modifier, Réimprimer, Remettre disponible
Perdu	Objet retrouvé	Modifier, Réimprimer, Objet retrouvé

Dialog	Script	API	Manager
Modifier l'objet	`article-edit.js`	`configure-article.php`	`configureArticle()`, `deleteArticleImages()`, `addArticleImages()`
Prêter / modifier un prêt	`article-loan.js`	`create-loan` / `update-loan`	`createLoan()` / `updateLoan()`
Enregistrer un retour	`article-return.js`	`return-loan.php`	`returnLoan()` + photos de dégâts
Marquer comme vendu	`article-sale.js`	`sell-article.php`	`sellArticle()`
Remettre disponible	`article-unsell.js`	`cancel-sale.php`	`cancelSale()`
Perdu / retrouvé	`article-lost.js`	`mark-lost` / `mark-found`	`markLost()` / `markFound()`
Réimprimer	`download-qr-modal.js`	`generate-qr-codes.php`	(TCPDF)
Contrat (lien)	—	`api/loan/contract.php`	(TCPDF)

Modifier l'objet

Le dialog lit le bloc JSON `#article-edit-data` (nom, catégorie, valeurs, photos, catégories avec leurs champs). Changer de catégorie redessine les champs en gardant ce qui a été tapé (`fieldValues`). Une photo retirée est seulement marquée (grisée, bouton ↺) et n'est supprimée qu'à l'enregistrement : le script envoie `delete_photos[]` (IDs) et `photos[]` (nouvelles photos réduites). Côté serveur, `deleteArticleImages()` ne supprime que des photos **de cet objet** (`WHERE idx_article = ? AND idx_degat IS NULL AND id_image IN (…)`), puis efface les fichiers après les lignes.

4.10 Prêt, retour, dégâts

Prêter

INTERFACE	JS	API	MANAGER	SQL
client (ou nouveau), sortie, retour prévu ou « indéterminée »	`article-loan.js` + `client-picker.js`	`api/loan/create-loan.php` (+ `includes/loan-form.php`)	`createClient()` si nouveau, `createLoan()`	`UPDATE articles` puis `INSERT INTO loans`

Le choix du client est un composant partagé (`includes/client-picker.php` + `window.clientPicker(form)`) : la liste des clients et « + Nouveau client », qui affiche prénom, nom et e-mail. Le retour prévu est pré-rempli avec la **durée par défaut** de l'organisation (`data-default-days`). Le nouveau client et le prêt sont créés dans la **même transaction** : si le prêt échoue, le client n'existe pas.

`LoanManager::createLoan() — le cœur de la protection contre les doubles prêts`

```
$ownTransaction = !$this->pdo->inTransaction();   // réutilise la transaction de l'API si elle existe
if ($ownTransaction) $this->pdo->beginTransaction();

$stmt = $this->pdo->prepare("UPDATE articles SET art_status = 'Emprunté'
    WHERE id_article = :article AND idx_organisation = :organisation AND art_status = 'Disponible'");
$stmt->execute([…]);
if ($stmt->rowCount() === 0) {                  // quelqu'un l'a prêté juste avant : aucune ligne modifiée
    throw new ArticleNotAvailableException(…);       // → l'API répond 409
}
INSERT INTO loans (idx_article, idx_client, lo_start_date, lo_due_date) VALUES (…);
```

Pourquoi des exceptions dédiées ?

`ArticleNotAvailableException`, `LoanAlreadyReturnedException`, `SaleNotCancellableException`, `LossConflictException`… permettent à l'API de distinguer un conflit normal (409, message clair) d'une vraie panne (500). Une `RuntimeException` ne suffirait pas : `PDOException` en hérite, toute erreur SQL serait prise pour un conflit.

Modifier un prêt en cours

Le bouton « Modifier » de l'historique ouvre le **même dialog** avec `data-loan-id`, `data-client-id`, `data-start-date`, `data-due-date` : le script passe en mode modification et appelle `update-loan.php`. `updateLoan()` remet le rappel à zéro si la date prévue change :

```
UPDATE loans SET lo_reminder_sent_at = IF(lo_due_date <=> :due_check, lo_reminder_sent_at, NULL),  -- avant lo_due_date !
    idx_client = :client, lo_start_date = :start, lo_due_date = :due
WHERE id_loan = :loan AND lo_end_date IS NULL
```

`<=>` est l'égalité de MySQL qui fonctionne avec `NULL` ; MySQL applique le `SET` de gauche à droite.

Enregistrer un retour

`article-return.js` envoie la date de retour et, si « Dégâts constatés » est choisi, une description et des photos. La fiche peut s'ouvrir avec `?retour=1` (bouton « Retour » des pages de prêts) : le dialog s'ouvre tout seul. `return-loan.php` fait tout dans une transaction :

```
UPDATE articles SET art_status = 'Disponible' WHERE … AND art_status = 'Emprunté';   -- rowCount 0 → 409
UPDATE loans SET lo_end_date = :end WHERE id_loan = :loan AND lo_end_date IS NULL;
INSERT INTO degats (idx_loan, deg_description) VALUES (…);          -- si dégâts
INSERT INTO images (im_name, idx_article, idx_degat) VALUES (…);    -- photos du dégât (addArticleImages)
```

Dans l'historique, un prêt avec dégâts a un bouton « Dégâts (n) » qui ouvre un dialog généré en PHP (description, date, photos).

4.11 Vente, perte, remise à disposition

Action	Ce que fait la méthode (transaction)
`sellArticle()`	`UPDATE … 'Vendu' WHERE art_status IN ('Disponible','Emprunté')` ; clôture le prêt en cours à la date de vente ; `INSERT INTO sales` . L'acheteur est l'emprunteur actuel par défaut (`data-borrower-id`).
`cancelSale()`	`UPDATE … 'Disponible' WHERE art_status = 'Vendu'` ; `UPDATE sales SET sa_cancelled_at = NOW()` . La vente est gardée ; `getLatestSale()` ignore les ventes annulées.
`markLost()`	`UPDATE … 'Perdu' WHERE art_status IN (…)` ; clôture le prêt en cours ; `INSERT INTO losses` (date, remarque visible des seuls membres).
`markFound()`	`UPDATE … 'Disponible' WHERE art_status = 'Perdu'` ; `UPDATE losses SET los_found_date = CURDATE()` .

Un objet vendu affiche « Vendu à … le … » ; un objet perdu affiche un bandeau rouge « Objet perdu depuis le … » avec la remarque.

4.12 Code QR scanné et objet perdu

La fiche accepte deux entrées : `?article_id=` (navigation interne) et `?code=` (adresse des codes QR, scannés par n'importe qui). Avec un code, la page n'exige pas d'être connecté d'emblée et décide :

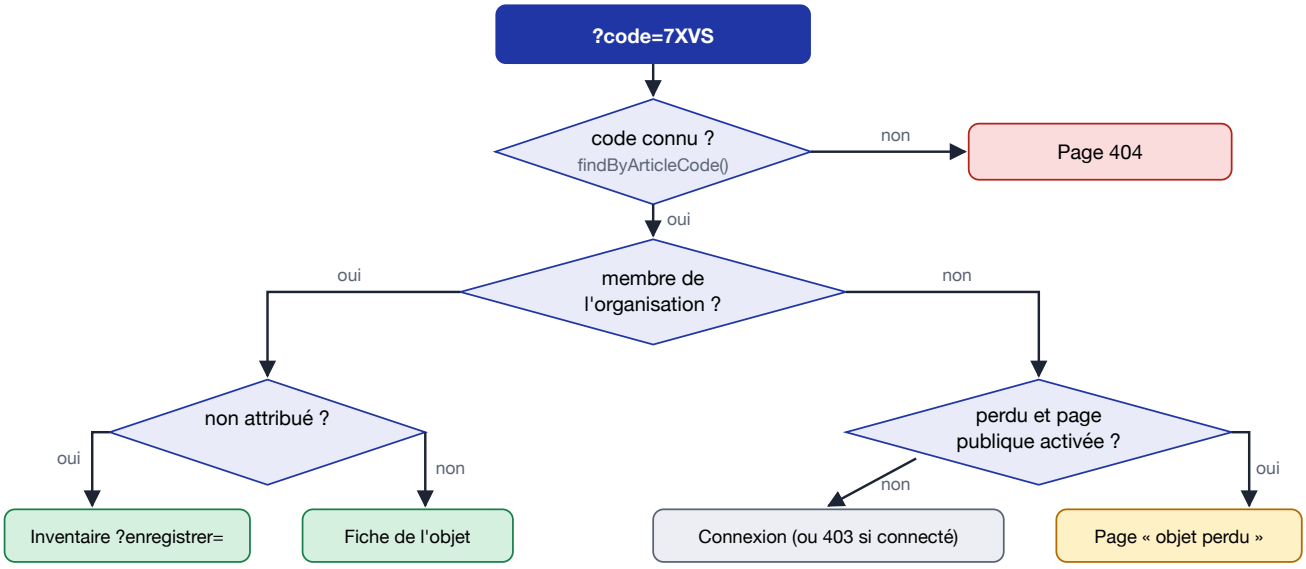


Figure 7 — Décision de la fiche quand elle est ouverte par un code QR. L'appartenance est vérifiée avec `getOrganisationById()` .

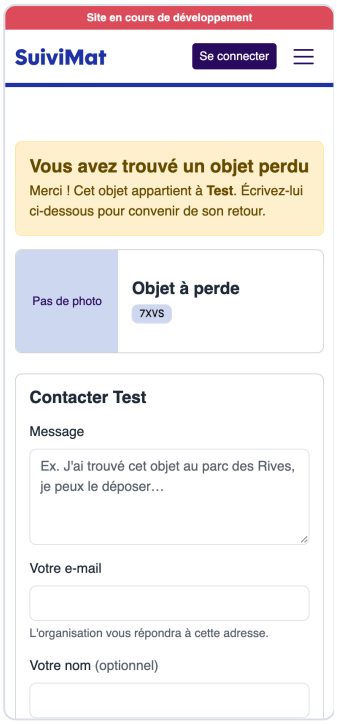
La page publique et le message

`includes/lost-article-public.php` ne montre que le nom, la photo, le code, le nom de l'organisation et son message (réglage « Objets perdus »). Rien de privé : ni remarque, ni historique, ni clients.

Le formulaire (`scripts/lost-contact.js`) envoie à `api/article/contact-lost.php`, **accessible sans connexion**, qui :

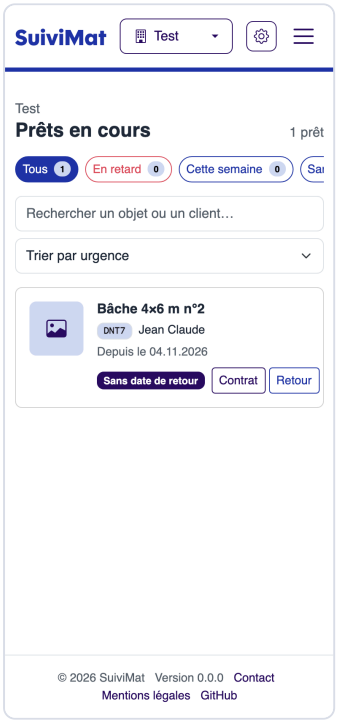
- ignore discrètement les robots (champ `website` caché rempli) ;
- limite à un message par minute (`$_SESSION['lost_contact_sent_at']`) ;
- refuse si l'objet n'est pas perdu ou si la page est désactivée ;
- envoie un e-mail à **chaque membre** (`getEmailsOfOrganisation()`) avec `Reply-To` = l'adresse de la personne, pour lui répondre directement.

`sendMail()` retire les retours à la ligne de l'adresse de réponse (sinon un visiteur pourrait ajouter des destinataires cachés : « injection d'en-têtes ») et encode le sujet en `=?UTF-8?B?…?=` pour les accents.

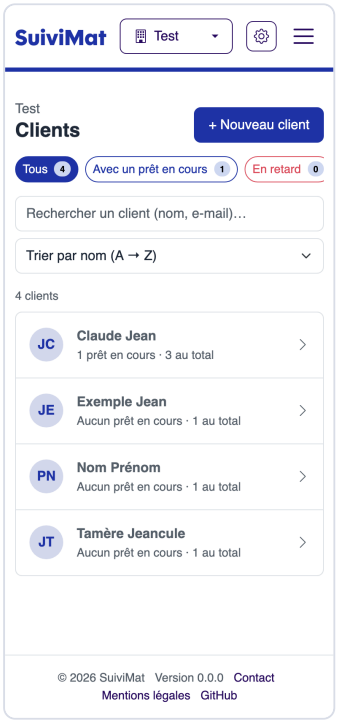


Page publique (non connecté)

4.13 Prêts en cours et clients



Prêts en cours



Clients



Page d'un client

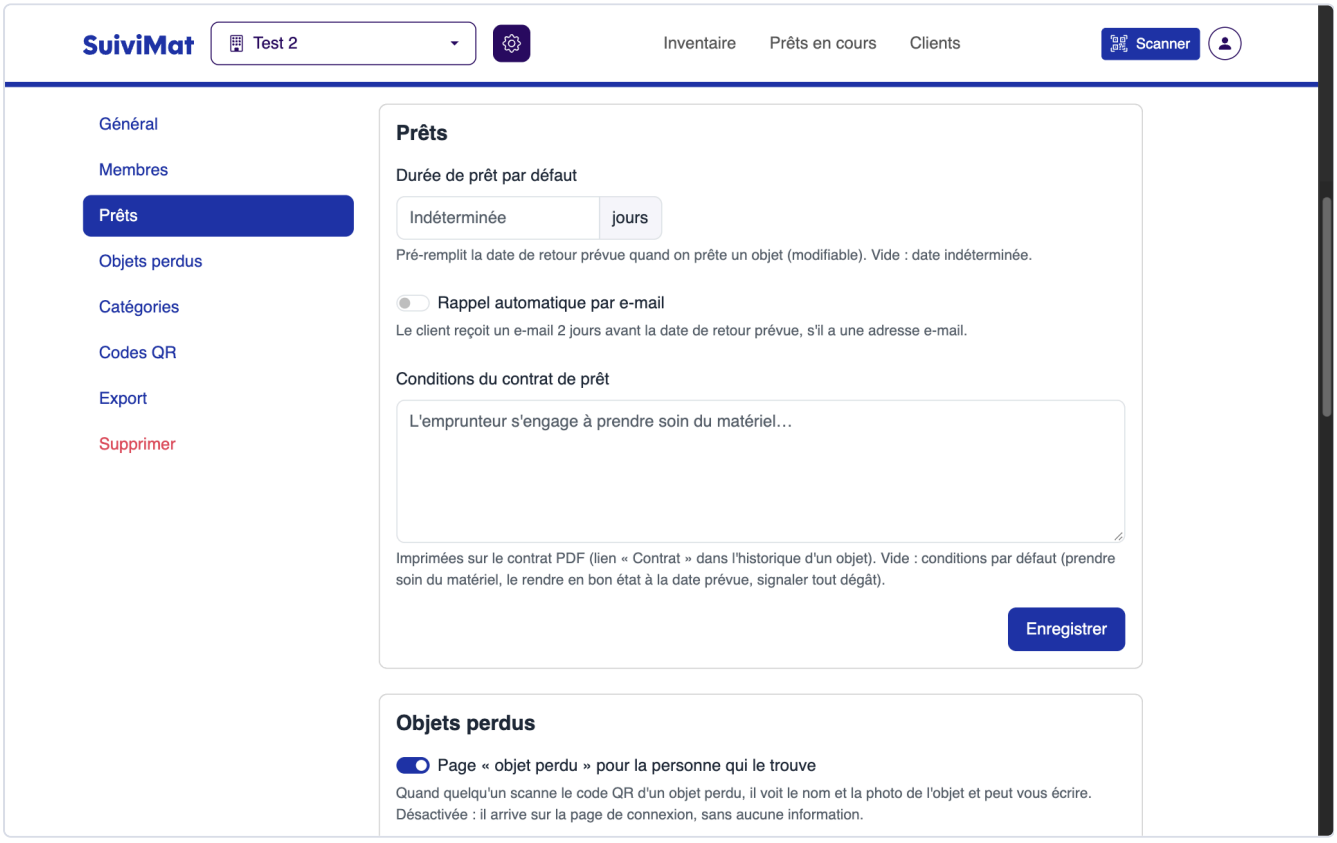
Ces trois pages sont générées entièrement en PHP ; le filtrage se fait ensuite **dans le navigateur**, sans requête, par `scripts/list-filter.js`. La page décrit chaque élément avec des attributs, et le script s'adapte :

```
<div data-list-filter>                                    <!-- zone gérée par list-filter.js -->
  <input type="radio" name="…" value="late">               <!-- [data-role=filters] -->
  <select data-role="sort"><option value="ongoing:desc">…   <!-- tri sur data-sort-ongoing, décroissant -->
  <a data-item data-categories="ongoing late" data-search="jean exemple …" data-sort-order="3" data-sort-ongoing="2">
```

Filtrer = masquer les éléments dont `data-categories` ne contient pas le filtre ou dont `data-search` ne contient pas chaque mot ; trier = replacer les éléments avec `list.append(item)` (un élément déjà présent est déplacé, pas copié).

- **Prêts en cours** : `getOngoingLoans()` (tri d'urgence en SQL : `ORDER BY lo_due_date IS NULL, lo_due_date, …`). Chaque carte est `includes/loan-card.php` ; le badge vient de `Loan::getDueStatus()` (`match (true)` : en retard, aujourd'hui, demain, dans N jours, sans date). Boutons « Contrat » et « Retour ».
- **Clients** : `getClientsWithLoanCounts()` compte en une requête les prêts de chaque client : `COUNT(loans.id_loan)`, `SUM(loans.lo_end_date IS NULL)` (en cours), `SUM(… AND lo_due_date < CURDATE())` (en retard), avec `LEFT JOIN` + `GROUP BY`.
- **Page d'un client** (`clients/client.php?client_id=`) : `getLoansForClient()`, séparés en cours / historique avec `array_filter()` ; « Modifier » ouvre `includes/modals/client-form-modal.php` → `update-client.php`.

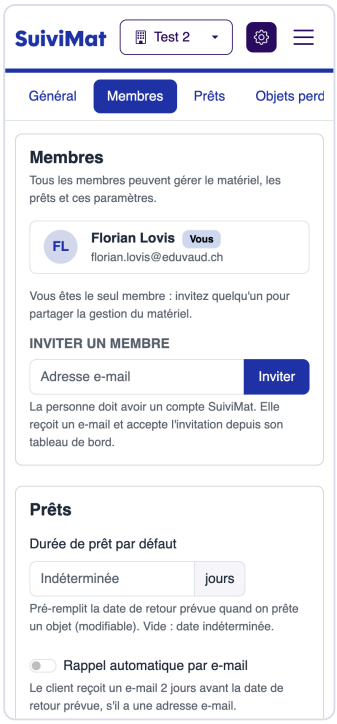
4.14 Paramètres de l'organisation



Paramètres sur ordinateur : sommaire collé à gauche, section visible en évidence.

`dashboard/organisation/settings/?id=`, ouvert par l'engrenage du header. Une carte par sujet ; `scripts/organisation-settings.js` envoie chaque formulaire `[data-role='settings-form']` à son API avec `organisation_id`.

Section	API → méthode
Général	`update-settings` (`section=general`) → `rename()`
Membres	`invite-member` → `invite()` + e-mail ; `cancel-invitation` ; `remove-member` (le dernier membre ne peut pas partir)
Prêts	`section=loans` → `updateLoanSettings()` : durée par défaut, rappels, conditions du contrat
Objets perdus	`section=lost` → `updateLostSettings()`
Catégories / QR / Export	modules existants (4.6, 4.4, 4.15)
Supprimer	`delete-organisation` → `deleteOrganisation()`



Mobile : pastilles collées sous le header

Quand une action recharge la page (invitation, retrait), le message de succès est gardé dans `sessionStorage` puis réaffiché dans la bonne section.

Le sommaire

Une seule `<nav id="settings-nav">` dans un `<aside class="col-lg-3">`. Les classes Bootstrap `nav-pills flex-nowrap flex-lg-column` la mettent en ligne (pastilles défilables) sur mobile et en colonne sur ordinateur. Un élément `position: sticky` ne reste collé que dans les limites de son parent :

- **mobile** : c'est l'`aside` qui colle (son parent, la `.row`, contient toute la page), avec un fond pour masquer les cartes qui passent dessous ;
- **ordinateur** : l'`aside` est une colonne aussi haute que la page, c'est la `nav` à l'intérieur qui colle.

La mise en évidence de la section visible est calculée par le script, pas par le ScrollSpy de Bootstrap : les dernières sections (Codes QR, Supprimer) sont trop courtes pour atteindre le haut de l'écran et ne seraient jamais actives.

`scripts/organisation-settings.js — updateActive() (appelée à chaque défilement)`

```
if (clickedIndex !== null) { setActive(clickedIndex); return; }   // lien cliqué : reste actif jusqu'au prochain défilement manuel
if (window.innerHeight + window.scrollY >= document.documentElement.scrollHeight - 4) {
    setActive(sections.length - 1); return;                       // bas de page : dernière section
}
const limit = window.innerHeight / 3;                              // dernière section passée au tiers supérieur
setActive(Math.max(0, sections.findLastIndex((s) => s.getBoundingClientRect().top <= limit)));
```

`clickedIndex` est remis à `null` par les événements `wheel`, `touchmove` et `keydown` (défilement fait par l'utilisateur). `setActive()` pose la classe `active` et `aria-current`, et sur mobile recentre la pastille dans la barre avec `nav.scrollTo()` (et non `scrollIntoView()`, qui ferait aussi bouger la page).

Supprimer une organisation

Il faut taper son nom exact. `deleteOrganisation()` exécute 14 requêtes dans une transaction, dans l'ordre imposé par les clés étrangères : images, dégâts, prêts, ventes, pertes, valeurs de champs, objets, champs, catégories (parentes détachées d'abord), clients, invitations, membres, organisation. La forme `DELETE degats FROM degats JOIN loans … JOIN articles … WHERE articles.idx_organisation = :id` supprime des lignes choisies grâce à une jointure. Les fichiers photo sont effacés après la validation.

4.15 Contrat PDF, export CSV, rappels

Contrat de prêt — GET `api/loan/contract.php?loan_id=`

Lien « Contrat » de l'historique et des cartes de prêt. Le contenu est écrit en HTML simple puis mis en page par TCPDF (`writeHTML()`) : organisation, emprunteur, objet (catégorie, informations), dates, conditions de l'organisation (ou texte par défaut), signatures.

Export de l'inventaire — GET `api/organisation/export-inventory.php?id=`

Une ligne par objet : code, nom, catégorie, statut, emprunteur, dates, informations. Le fichier commence par le caractère « BOM » UTF-8 (sinon Excel affiche mal les accents) et utilise le **point-virgule**, séparateur attendu par Excel en Suisse et en France (`fputcsv($output, $row, ';')`).

Rappels automatiques — CLI `cron/send-loan-reminders.php`

Script à lancer chaque jour par une tâche planifiée (cron). Il refuse de s'exécuter depuis un navigateur (`PHP_SAPI !== 'cli'`) et le dossier est bloqué par un `.htaccess`.

```
SELECT … FROM loans JOIN articles … JOIN clients … JOIN organisations …
WHERE loans.lo_end_date IS NULL AND loans.lo_reminder_sent_at IS NULL
  AND loans.lo_due_date BETWEEN CURDATE() AND CURDATE() + INTERVAL 2 DAY   -- rattrape un jour manqué
  AND organisations.org_reminders_enabled = 1 AND clients.cli_email IS NOT NULL;
-- pour chaque prêt : sendMail(client) puis
UPDATE loans SET lo_reminder_sent_at = NOW() WHERE id_loan = :loan;
```

En local : `SERVER_NAME=localhost php cron/send-loan-reminders.php` (les e-mails sont écrits dans le terminal, pas envoyés).

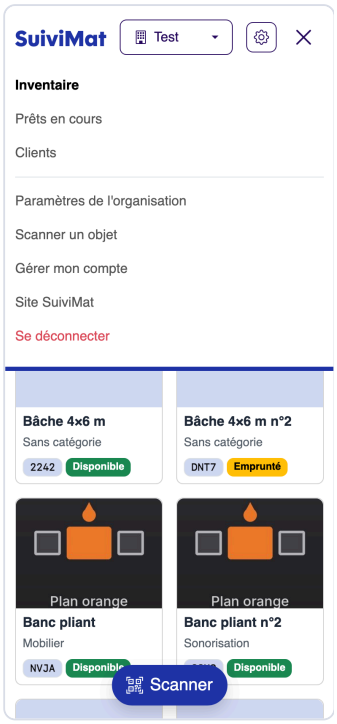
5 Le front-end transversal

5.1 Header, footer et mise en page

`includes/app-header.php` (application) contient : logo (→ tableau de bord), sélecteur de l'organisation courante (toutes les organisations de l'utilisateur), engrenage des paramètres, liens Inventaire / Prêts en cours / Clients, bouton Scanner et menu du compte. Sur mobile, liens et compte passent dans le menu burger. Si la page a une `$organisation`, le header inclut aussi le dialog du scanner rapide.

Le lien actif est mis en gras quand le fichier de la page (`$_SERVER['SCRIPT_NAME']`) commence par un des chemins de sa section (`str_starts_with()`) : la page d'un client active « Clients ».

`includes/app-footer.php` : une ligne (copyright, version, contact, mentions légales, GitHub). Les éléments fixés en bas sur mobile (barre d'actions, bouton Scanner) réservent de la place avec `<body class="has-action-bar">`.



Menu burger ouvert

5.2 CSS : Bootstrap, Sass et variables

- `style/scss/style.scss` importe **Bootstrap 5** entier après avoir redéfini ses variables (couleurs, police), puis ajoute les styles du projet. Compilation : `cd style && npm run build` → `style/css/style.css`.
- **Hauteur du header** : variable **CSS** (et non Sass, calculée une seule fois à la compilation) qui change selon la page :

```
:root    { --header-height: 96px; }   /* site public (bandeau + barre) */
body.app { --header-height: 79px; }   /* application */
body     { padding-top: var(--header-height); display: flex; flex-direction: column; min-
height: 100vh; }
footer   { margin-top: auto !important; }   /* footer poussé en bas des pages courtes */
```

- **Footer en bas** : `body` est une colonne flex d'au moins la hauteur de l'écran ; `margin-top: auto` donne au footer toute la place libre au-dessus de lui. Ne jamais mettre de classe `mt-*` sur un `<footer>` : plus spécifique, elle annulerait cette règle.
- **Cache** : CSS et scripts sont inclus avec `asset()` (2.1) ; après `npm run build`, le navigateur prend automatiquement la nouvelle version.

5.3 Scripts partagés

Fichier	Rôle
`sendForm.js`	`sendForm(form, url, options)` : envoi d'un formulaire, message, redirection (connexion, inscription, création d'organisation).
`scanner.js`	Classe `Scanner` (4.8).
`list-filter.js`	Filtres / recherche / tri des listes déjà affichées (4.13).
`resize-photo.js`	`window.resizePhoto(file)` : photo réduite en JPEG avant l'envoi.
`category-fields.js`	`window.categoryFields(id, categories)` : champs d'une catégorie avec ceux de ses parentes.
`client-picker.js`	`window.clientPicker(form)` : liste de clients + « Nouveau client ».
`quick-scan-modal.js`	Scanner rapide (4.8).
`download-qr-modal.js`	Dialog des codes QR (nouveaux / réimpression).

> **Fonctions « globales » en JavaScript**
>
> Les scripts partagés accrochent leur fonction à `window` (`window.resizePhoto = …`) pour qu'elle soit utilisable par les autres scripts, eux-mêmes enfermés dans leur fonction. Ils doivent être inclus **avant** les scripts qui les utilisent : avec `defer`, les scripts s'exécutent dans l'ordre de la page.

6 Annexes

6.1 Toutes les API

Toutes exigent d'être connecté et membre de l'organisation concernée, sauf mention contraire. Les réponses sont en JSON (`success` , `message` en cas d'erreur).

Point d'entrée	Paramètres	Effet
POST `auth/login/submit-login.php`	email, password, redirect	Connexion (public)
POST `auth/register/submit-register.php`	prénom, nom, e-mail, mots de passe, captcha	Inscription + connexion (public)
POST `api/account/update-profile.php`	firstname, lastname, email, current_password	Informations personnelles
POST `api/account/update-password.php`	current, new, confirm _password	Mot de passe
POST `api/account/delete-account.php`	password	Supprime le compte
GET `api/article/get-articles.php`	organisation_id, page, categorie, statut, q	Inventaire paginé + compteurs
GET `api/article/search-suggestions.php`	organisation_id, q, statut	Propositions de recherche
GET `api/article/get-article.php`	organisation_id, code	Un objet (scanner, enregistrement)
POST `api/article/configure-article.php`	organisation_id, code, name, category_id, fields[], photos[], delete_photos[]	Configure / modifie un objet
POST `api/article/contact-lost.php`	code, email, message, name, phone, website	Message pour un objet perdu (**public**)
GET / POST `api/category/*.php`	get / create / update / delete-category	Catégories et champs
GET / POST `api/client/*.php`	get / create / update-client	Clients
POST `api/invitation/respond-invitation.php`	invitation_id, action	Accepter / refuser
POST `api/loan/create-loan.php`	article_id, client_id (ou new + nom), start_date, due_date	Prêter
POST `api/loan/update-loan.php`	+ loan_id	Modifier un prêt en cours
POST `api/loan/return-loan.php`	article_id, end_date, damaged, damage_description, photos[]	Retour (+ dégâts)
GET `api/loan/contract.php`	loan_id	Contrat PDF

POST `api/loss/mark-lost.php` / `mark-found.php`	article_id, loss_date, comment	Perdu / retrouvé
POST `api/sale/sell-article.php` / `cancel-sale.php`	article_id, client_id, sale_date	Vendre / remettre disponible
POST `api/organisation/create-organisation.php`	organisation_name	Nouvelle organisation
POST `api/organisation/generate-qr-codes.php`	export_type, mode, object_count, reprint_*, format	Génère le PDF, renvoie `{ url }`
GET `api/organisation/qr-pdf.php`	token	Sert le PDF généré, avec son nom de fichier
POST `api/organisation/update-settings.php`	section + champs	Réglages
POST `api/organisation/invite-member.php`, `cancel-invitation`, `remove-member`	email / invitation_id / user_id	Membres
POST `api/organisation/delete-organisation.php`	confirm_name	Supprime tout
GET `api/organisation/export-inventory.php`	id	CSV de l'inventaire

6.2 Mise en ligne

1. Exécuter les migrations de `database/migrations/` (3.4).
2. Définir les variables d'environnement : `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `RECAPTCHA_SECRET_KEY`.
3. `composer install` (TCPDF dans `vendor/`) et `cd style && npm install && npm run build`.
4. Vérifier que `mail()` peut envoyer des e-mails depuis le serveur.
5. Créer la tâche planifiée : `0 8 * * * cd /chemin/du/site && php cron/send-loan-reminders.php`.
6. Régénérer les étiquettes déjà imprimées avec l'ancienne adresse `/organisation/article/?code=` (réimpression, 4.4).

6.3 Limites connues

- **Mentions légales** : la section « Services externes » n'indique pas **Google reCAPTCHA**, utilisé à l'inscription. À corriger avant la mise en ligne.
- Pas de « mot de passe oublié » ; pas de suppression de client.
- Dates calculées en **UTC** : retards et échéances peuvent avoir un jour de décalage entre minuit et 1-2 h en Suisse.
- Pas de limitation des essais de connexion.
- On n'invite que des personnes ayant déjà un compte.
- L'historique d'un objet n'affiche que les prêts (pas les ventes ni les pertes).

- La photo de catégorie (`cat_img_name`) existe dans la base mais n'est pas utilisée.